

AI Accelerator for TinyML with RISC-V Architecture

Complete Project Guide — 4-Week Summer Training (AICTE Idea Lab)

Team size: 5 members | **Duration:** 4 weeks | **Board:** FPGA (provided by lab) | **Approach:** Software model first, then hardware (RTL → FPGA)

1. What you are actually building

In plain terms: you are designing a small, custom "math helper" circuit (an accelerator) that plugs into an open-source RISC-V processor. Normal processors do AI math (multiply-accumulate operations, the core of neural networks) one step at a time. Your accelerator does many of these steps in parallel, in dedicated hardware, so a tiny neural network (TinyML model — e.g. weather prediction, machine vibration/health anomaly detection) runs faster and on far less power. You will first prove the idea works in software (Python/C simulation), then implement it as real digital hardware (Verilog/SystemVerilog), and finally load it onto an FPGA board to test it live.

This is essentially a mini "CPU + custom coprocessor" system-on-chip design project.

2. Prerequisites — what you should know or refresh before/at the start

You don't need to master everything below individually — split by role (Section 7), but everyone should understand the basics.

Area	What to know	Why
Digital logic fundamentals	Logic gates, flip-flops, registers, FSMs, combinational vs sequential circuits	This is the language hardware is designed in
Computer architecture basics	What a CPU does: fetch-decode-execute cycle, registers, ALU, instruction sets	You are extending a CPU, so you must understand how one works
C programming	Functions, pointers, arrays, fixed-point math	Used for the software model and for programs that run on the RISC-V core
Python basics	NumPy, basic scripting	Used for the software/algorithm-level accelerator model and TinyML training
Basic linear algebra	Matrix multiplication, dot products	Neural network inference is mostly multiply-accumulate (MAC) operations
Command line / Linux basics	Navigating folders, running scripts, installing packages	All the tools below are Linux-friendly (or run via WSL on Windows)
Git/GitHub basics	Clone, commit, push, branches	Needed for 5-person team collaboration

None of this needs to be expert-level going in — the first week of the training is meant to build these up.

3. New concepts you'll need to learn (the "things you don't know yet")

1. **RISC-V ISA (Instruction Set Architecture)** — an open-source, free instruction set (unlike ARM/x86 which are proprietary). You'll learn its base instruction set (RV32I) and how it allows **custom instruction extensions**.
2. **Custom instruction extensions** — RISC-V reserves specific opcode space for custom instructions. You will literally define a new instruction like `MAC.AI rd, rs1, rs2` that your accelerator hardware understands.
3. **HDL (Hardware Description Language)** — Verilog or SystemVerilog. This is how you describe circuits as code. It looks like programming but describes physical hardware, not sequential software.
4. **RTL (Register Transfer Level) design** — the level of abstraction where you describe how data moves between registers each clock cycle. This is what you'll actually write for the accelerator.

5. **TinyML / quantized neural networks** — full-size AI models are too big for microcontrollers. TinyML uses small, quantized (8-bit integer instead of 32-bit float) models so they fit in kilobytes of memory.
 6. **MAC arrays / systolic arrays** — the actual "expressway" hardware structure: a grid of small multiply-accumulate units that process many numbers in parallel. This is the heart of almost every AI accelerator (from Google TPU down to tiny edge chips).
 7. **FPGA vs ASIC** — an FPGA (Field-Programmable Gate Array) is reconfigurable hardware — you can "burn" your Verilog design onto it and change it as many times as needed. An ASIC is a permanent chip. You're using FPGA because it's testable and reprogrammable — a real chip would be the final industrial step (out of scope here, but good to mention in your report).
 8. **Simulation vs synthesis vs implementation** — three different stages of hardware design: simulation checks logic is correct (no real hardware involved), synthesis converts your Verilog into logic gates, implementation maps those gates onto the actual FPGA's physical resources.
 9. **Testbenches and waveform verification** — how hardware designers "test" their code, by feeding inputs and checking outputs on a timing diagram, since you can't just `print()` debug hardware the way you would software.
 10. **AXI / Wishbone bus basics** — standard on-chip communication protocols used to connect your CPU core to your custom accelerator and memory. You'll use a simplified version of one of these.
-

4. Software tools you will need

Simulation / software modeling (Week 1-2)

- **Python 3 + NumPy** — build a functional (non-hardware) model of your accelerator's math, and a small TinyML model.
- **TensorFlow Lite for Microcontrollers** or a hand-written tiny neural net in NumPy — for the weather/machine-health model.
- **Jupyter Notebook** — good for quick experimentation and visualizing results with your team.

RISC-V toolchain

- **RISC-V GNU Toolchain** (`riscv32-unknown-elf-gcc`) — compiles C code into RISC-V machine code that will run on your core.
- **Spike** (RISC-V ISA simulator) or **QEMU (RISC-V target)** — lets you run/debug RISC-V programs without any hardware, useful before you even touch the FPGA.

Hardware design (RTL)

- **A soft RISC-V core** — don't write a CPU from scratch (too big for 4 weeks). Use an

existing open-source core and extend it:

- **PicoRV32** — small, simple, well documented, easiest to modify for custom instructions. **Recommended for your timeline.**
- **VexRiscv** — more configurable but written in SpinalHDL (steeper learning curve).
- **Rocket Chip / CVA6** — powerful but too complex for a 4-week team project.
- **Verilog / SystemVerilog** — the HDL you'll write your accelerator module and instruction-decode logic in.
- **Icarus Verilog + GTKWave** — free, lightweight simulator + waveform viewer. Use this for day-to-day RTL debugging before touching the FPGA tools (much faster iteration).
- **Verilator** (optional, faster simulation for larger designs) — good if your team wants faster simulation cycles later.

FPGA toolchain (depends on what board the lab gives you — check this first)

- **Xilinx Vivado** — if it's a Xilinx/AMD board (e.g. Artix-7, Zynq, Basys 3/Nexys boards are common in labs).
- **Intel Quartus Prime** — if it's an Intel/Altera board (e.g. DE10-Lite, Cyclone).
- **Open-source alternative: Yosys + nextpnr** (works for Lattice iCE40/ECP5 boards) — fully open-source flow, good learning tool if your board supports it.

Ask the lab exactly which FPGA board and vendor toolchain they provide on day one — this determines your entire hardware-side workflow.

Team collaboration

- **GitHub/GitLab** — shared repository, one branch per subsystem, pull requests reviewed by teammates.
- **VS Code** with Verilog/HDL extensions — common editor across the team.
- **Draw.io / Excalidraw** — for architecture diagrams and planning (in addition to what's shown above).
- **Google Docs / Notion** — shared log of decisions, test results, and the weekly plan.

5. How the pieces connect (system overview)

The block diagram above shows the core relationships: your **host PC** programs the **FPGA board** over USB/JTAG. Inside the FPGA, the **RISC-V CPU core** talks to your **custom AI accelerator unit** over a shared **bus**, and both can read/write **on-chip memory** holding the neural network's weights and incoming sensor data. A **sensor** (or simulated sensor input, since real weather/vibration sensors are optional extras) feeds data in, and the result of inference drives an **output** — an LED, UART message to a serial monitor, or small display.

The key idea to explain in your report: normally, the CPU alone would compute every multiply-accumulate operation of the neural network one at a time using ordinary instructions (`ADD`, `MUL`, load, store — many cycles per operation). With the custom instruction and accelerator, the CPU instead issues **one instruction** that tells the accelerator "process this whole block of MACs," and the parallel hardware finishes it in far fewer cycles, using less energy per operation.

6. Four-week plan (software → hardware → FPGA)

The workflow diagram above shows the six phases; here is how they map onto your 4 weeks (5-person team, ~2 people working in a "pair" per subsystem, rotating so everyone touches both software and hardware sides).

Week 1 — Foundations + software model

- Days 1-2: Crash-course RISC-V ISA, Verilog basics, TinyML basics (whole team together, use lab sessions/tutorials).
- Days 3-4: Everyone sets up tools (RISC-V GCC toolchain, Spike/QEMU, Icarus Verilog, Vivado/Quartus, Git repo).
- Days 5-7: Build the **software-only model**: a small TinyML model (weather or vibration classifier) in Python/NumPy, and a NumPy function that mimics what your future hardware accelerator will do (batched MAC operations). Measure baseline: how many operations/cycles the "naive" approach takes.
- **Deliverable**: working Python simulation + report on expected speedup from parallelizing MACs.

Week 2 — RTL design (the actual hardware description)

- Design the **custom instruction** (opcode format, operands) that will trigger the accelerator.
- Modify PicoRV32 (or chosen core) to recognize this instruction and route operands to your accelerator module.
- Write the **accelerator RTL** — a small MAC array (start with something modest: 4-8 parallel MAC units is a very reasonable and impressive scope for 4 weeks, not hundreds).
- Design memory/register interface so accelerator can read input data and weights.
- **Deliverable**: RTL code for core modification + accelerator module (not yet tested).

Week 3 — Simulation, verification, and FPGA bring-up

- Days 1-3: Write **testbenches** in Icarus Verilog/GTKWave. Simulate the modified core executing your custom instruction and check waveforms match expected MAC results from your Week 1 Python model (this cross-check is your strongest proof of correctness).

- Days 4-5: Fix bugs found in simulation (expect several iterations — this is normal).
- Days 6-7: **First FPGA synthesis** — synthesize the design in Vivado/Quartus, resolve any synthesis-only errors (timing, resource usage), and program the board. Confirm the base RISC-V core boots and runs a simple test program on real hardware.
- **Deliverable:** simulation waveforms proving correctness + first successful FPGA bitstream load.

Week 4 — Full integration, benchmarking, and reporting

- Days 1-2: Load your actual TinyML model's weights into FPGA memory, run real inference on the board using your custom instruction end-to-end (sensor/test-vector input → accelerator → output).
 - Days 3-4: **Benchmark:** measure clock cycles/latency and, if possible, power (some boards have onboard power measurement headers, or estimate from Vivado's power report) — compare against running the same inference using ordinary instructions only (no accelerator). This comparison is the entire point of your project — make sure you capture clear before/after numbers.
 - Days 5-6: Polish demo (LED/UART/display output), prepare slides, finalize report and block diagrams.
 - Day 7: Final presentation/demo to the lab.
 - **Deliverable:** working FPGA demo + benchmark data + final report and presentation.
-

7. Suggested role split for a 5-person team

Rotate people through these roughly weekly so everyone gets both software and hardware exposure — but having clear primary owners avoids five people editing the same file.

1. **TinyML/algorithm lead** — builds and quantizes the Python neural net model, defines what "AI math" the accelerator must accelerate.
2. **RISC-V core lead** — works inside PicoRV32, adds the custom instruction decode logic, owns the CPU-side integration.
3. **Accelerator RTL lead** — designs and writes the MAC array/accelerator Verilog module itself.
4. **Verification lead** — writes testbenches, runs simulations, tracks bugs, maintains the "software model vs hardware result" comparison.
5. **FPGA/toolchain + integration lead** — owns Vivado/Quartus project, synthesis, timing closure, board bring-up, and the final demo/benchmarking; also keeps the Git repo and weekly plan organized.

All five should jointly own the report and presentation — each explains their own subsystem.

8. Resources to study from

- **RISC-V:** "The RISC-V Reader" (free PDF, Patterson & Waterman) and the official riscv.org specifications.
 - **PicoRV32 GitHub repository** (by YosysHQ/Clifford Wolf) — read its README and source; it's small enough to read in full.
 - **Verilog:** ASIC-World Verilog tutorial (free, beginner-friendly) or Nandland's Verilog tutorials.
 - **TinyML:** "TinyML" by Pete Warden & Daniel Situnayake (O'Reilly) — the standard intro text; also look at **TensorFlow Lite for Microcontrollers** examples.
 - **CFU Playground** (Google) — an excellent reference open-source project that does almost exactly this (custom accelerator instructions for TinyML on FPGA with a RISC-V core). Very worth browsing for design inspiration, even if you don't use it directly.
 - **Vivado/Quartus:** official "getting started" guides from Xilinx/Intel for your specific board model.
-

9. Practical tips / things that trip teams up

- **Confirm the exact FPGA board model on day one** — the toolchain (Vivado vs Quartus vs open-source) and available resources (logic cells, block RAM) depend entirely on this, and it changes what's realistic for your accelerator size.
 - **Keep the accelerator small at first** (e.g., a 4-wide or 8-wide MAC array). It's far better to have a small, fully working, well-benchmarked accelerator than an ambitious one that never boots on real hardware. You can mention "scalable to larger arrays" as future work.
 - **Always verify in simulation before synthesizing.** FPGA synthesis + programming cycles are slow (minutes) compared to Icarus Verilog simulation (seconds) — debug logic in simulation first.
 - **Cross-check hardware output against your Week 1 Python model.** This is your proof of correctness and makes for a strong report.
 - **Version control discipline matters with 5 people editing Verilog** — uncommitted or conflicting RTL changes are the #1 source of "it worked yesterday" bugs. Commit often, use branches.
 - **Document power/performance numbers carefully** — the "ultra-low power" claim is the whole selling point of the project, so having real, defensible cycle-count and (if possible) power numbers will make your final presentation much stronger than the working demo alone.
-

10. Final deliverables checklist

- ☐ Python/NumPy software model of the TinyML accelerator math, with baseline performance numbers
- ☐ Custom RISC-V instruction specification (opcode, operand format)
- ☐ Modified RISC-V core RTL (PicoRV32 + custom decode logic)
- ☐ Accelerator RTL module (MAC array)
- ☐ Testbenches + simulation waveforms proving correctness
- ☐ Successful FPGA synthesis, implementation, and bitstream programming
- ☐ Working live demo on FPGA board (sensor/test input → inference → output)
- ☐ Benchmark comparison: accelerated vs non-accelerated (cycles, and power if measurable)
- ☐ Architecture diagrams (block diagram, workflow diagram — see above)
- ☐ Final report + presentation slides